
Warranty GENE — Shrish task list

Scope of this document. Shrish provisions the Azure database that will hold the Warranty GENE dealer records, loads the JSON records we hand him, runs the tests we wrote, and reports pass or fail. That is the paid scope.

Common sense allowed — fork first. These instructions were written quickly. If Shrish spots something worth improving — a smarter index, a cleaner column, a better ingestion pattern — he is free to do it. The only rule: **fork the source repository first, and make the change on the fork**. Do not push directly to `EVEglyphDesign/hawkins-twin-platform`. That way Dany can see the change on a surface, catch up with Shrish, and decide what to merge upstream. Nothing gets slowed down waiting for approval; nothing gets lost inside a working session either.

This document is adaptive, not fixed. Warranty GENE is designed to be redirected as we learn. Dany may loosen a rule, tighten another, or replace whole tasks between one working session and the next. When a redirect happens, the document is reissued at the same canon URL with a new revision line in the provenance block, and the fork initiatives from the previous revision are folded into the discussion. Do not treat any single revision as the final word. Do treat the current revision as the operating contract until a new one is published.

Canon rule (given, not to be re-decided)

- The **VIN is the base code** for every equipment record. Every table's primary anchor column is `vin`, every filename in the source is `{VIN}.json`, every index is keyed by VIN. This one is load-bearing across every downstream ring, so keep it as the primary key. If you have a strong reason to add a surrogate id alongside it on your fork, that's a fork discussion.
- The record shape is at `warranty-gene/schema/rich-target.schema.json`, and the sample records at `warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins/` are the canonical shape. Load them as-is for the first pass; anything you want to change goes on the fork.

What Shrish does

1. Provision the Azure resources listed in Task 1 with the sizing listed in Task 2.
2. Load the seven sample VIN records from the source repository into the database, unchanged, as Task 3.
3. Run the four verification queries in Task 4 and record pass or fail against the expected values Task 4 states.
4. Repeat Task 3 and Task 4 exactly when the first real dealer extract lands from PACCAR (arriving after the credentials in `EgD-WGB-003`).
5. Report back with the four rows Task 5 asks for. Nothing else.

Where the line is

Two rules, everything else is fair game with common sense:

1. **Do not push to `EVEglyphDesign/hawkins-twin-platform` directly.** If you want to change anything in the source repository — schema, scripts, DDL, samples, this document — fork it, change it on the fork, and mention the fork URL in the Task 5 report. Dany will pull what makes sense.
 2. **Do not lose the VIN as the primary key** and do not reshape a record such that the source JSON file cannot round-trip back out of the database. Everything else — extra columns, extra indexes, materialized views, a helper script, a better ingestion pattern — is fine, as long as it lives on the fork.
-

Task 1 · Provision the Azure database

Choice. Azure Database for PostgreSQL Flexible Server, or Azure SQL Database. Shrish picks one and does not switch after Task 3.

Provision, exactly.

- One database named `warranty_gene` .
- One schema named `dealer_records` .
- One service-account login named `warranty_gene_loader` , password stored in Azure Key Vault, no personal Azure AD account attached to the database.
- Firewall: allow Shrish's workstation IP only, until Dany hands over a second IP.
- Backup: default point-in-time restore, 7 days, no custom retention policy.

Gate. `SELECT 1;` returns from `warranty_gene_loader` against the `warranty_gene` database, over TLS, from Shrish's workstation. Send Dany the resource id (the full `/subscriptions/.../databases/warranty_gene` path) and nothing else.

Task 2 · Create the one table and the four indexes

Use exactly this DDL. Do not change column names. Do not change indexes. Do not add columns. If it does not run, the driver or the permissions are wrong — fix those, not the DDL.

If Postgres

```
CREATE TABLE dealer_records.vin_record (
  vin          TEXT PRIMARY KEY,
  as_of        TIMESTAMPTZ NOT NULL,
  rooftop_code TEXT NOT NULL,
  payload      JSONB NOT NULL,
  ingested_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE INDEX ix_vin_record_rooftop      ON dealer_records.vin_record (rooftop_code);
CREATE INDEX ix_vin_record_model        ON dealer_records.vin_record ((payload ->
'chassis' ->> 'model'));
CREATE INDEX ix_vin_record_in_service   ON dealer_records.vin_record ((payload ->
'chassis' ->> 'in_service_date'));
CREATE INDEX ix_vin_record_score_total  ON dealer_records.vin_record (((payload ->
'score' ->> 'total')::numeric));
```

If Azure SQL

```
CREATE TABLE dealer_records.vin_record (
  vin          NVARCHAR(17) NOT NULL PRIMARY KEY,
  as_of        DATETIME2 NOT NULL,
  rooftop_code NVARCHAR(16) NOT NULL,
  payload      NVARCHAR(MAX) NOT NULL CHECK (ISJSON(payload) = 1),
  ingested_at  DATETIME2 NOT NULL DEFAULT SYSUTCDATETIME()
);

CREATE INDEX ix_vin_record_rooftop      ON dealer_records.vin_record (rooftop_code);
CREATE INDEX ix_vin_record_model        ON dealer_records.vin_record
  (CAST(JSON_VALUE(payload, '$.chassis.model') AS NVARCHAR(32)));
CREATE INDEX ix_vin_record_in_service   ON dealer_records.vin_record
  (CAST(JSON_VALUE(payload, '$.chassis.in_service_date') AS DATE));
CREATE INDEX ix_vin_record_score_total  ON dealer_records.vin_record
  (CAST(JSON_VALUE(payload, '$.score.total') AS DECIMAL(9,2)));
```

Gate. All four CREATE INDEX statements succeed. SELECT COUNT(*) FROM dealer_records.vin_record; returns 0. Send Dany the two lines: engine name (Postgres or SQL) and the count.

Task 3 · Load the seven sample VIN records, unchanged

Source. Every *.json file under `warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins/` on main. Download via the GitHub raw URL for each file. Do not clone the whole repository. Do not read from a local copy; read the raw files from main at the time of load.

Method. For each file, read the JSON, and insert one row: `vin` from the JSON's `vin` field, `as_of` from `as_of`, `rooftop_code` set to `R00-sample` (Shrish does not derive this from the record; it comes from the source directory name, which is why the loader is a single line, not a rule engine), `payload` set to the entire JSON document unchanged.

Rule. For the first pass, the record on disk is the record in the database — whole JSON, VIN preserved as-is, no field rewrites. This makes the round-trip test in Task 4 a real test. If you want to store additional derived columns alongside the payload on your fork, that's fine; keep the untouched `payload` too.

Gate.

```
SELECT COUNT(*) FROM dealer_records.vin_record;
-- expected: 7
```

Send Dany the exact number returned.

Task 4 · Run the four verification queries

Each query has an expected answer produced from the source records. If the returned value does not match, the load in Task 3 is wrong — fix Task 3 and re-run, do not "adjust" the query.

4.1 · Every VIN loaded

```
SELECT vin FROM dealer_records.vin_record ORDER BY vin;
```

Expected, exactly.

```
1XPBSYN00L1000404
1XPBSYN00M1000303
1XPBSYN00N1000101
1XPBSYN00N1000707
1XPBSYN00P1000202
1XPBSYN00P1000606
1XPBSYN00R1000505
```

4.2 · Model breakdown

Postgres:

```
SELECT payload -> 'chassis' ->> 'model' AS model, COUNT(*) AS n
FROM dealer_records.vin_record
GROUP BY 1 ORDER BY 1;
```

Azure SQL:

```
SELECT JSON_VALUE(payload, '$.chassis.model') AS model, COUNT(*) AS n
FROM dealer_records.vin_record
GROUP BY JSON_VALUE(payload, '$.chassis.model')
ORDER BY 1;
```

Expected, exactly.

model	n
220EV	1
537	1
567	1
579	4

4.3 · Coverage-bucket key count on record 1XPBSYN00N1000101

Postgres:

```
SELECT jsonb_object_keys(payload -> 'coverage')
FROM dealer_records.vin_record
WHERE vin = '1XPBSYN00N1000101';
```

Azure SQL:

```
SELECT [key]
FROM dealer_records.vin_record
CROSS APPLY OPENJSON(JSON_QUERY(payload, '$.coverage'))
WHERE vin = '1XPBSYN00N1000101';
```

Expected. Ten rows, with these keys, in any order: base_emissions_carb, base_emissions_epa_eccc, standard_vehicle, standard_mx_engine, engine_major_components, vehicle_major_components, severe_service, paccar_alternator_starter, paccar_front_axles, non_paccar_front_axles.

4.4 · Highest score total

Postgres:

```
SELECT vin, (payload -> 'score' ->> 'total')::numeric AS total
FROM dealer_records.vin_record
ORDER BY total DESC
LIMIT 1;
```

Azure SQL:

```
SELECT TOP 1 vin, CAST(JSON_VALUE(payload, '$.score.total') AS DECIMAL(9,2)) AS total
FROM dealer_records.vin_record
ORDER BY 2 DESC;
```

Expected. Exactly one row. Send Dany the VIN and the number. Dany will verify against the source; Shrish does not need to know the expected value in advance.

Task 5 • Report back

Message Dany with these four rows, in this order:

1. **Engine.** Postgres flexible server or Azure SQL , plus the region.
2. **Row count.** The single number from Task 3's gate.
3. **Model breakdown match.** pass or fail for Task 4.2 against the expected table. If fail , include the actual table.
4. **Top-score row.** VIN and total from Task 4.4.

Optional fifth row — include it if it applies:

1. **Fork initiatives.** URL of your fork and a one-line summary of anything you changed or added — extra columns, an extra index, a helper script, a suggested edit to this document. One line per initiative. Dany will walk through them with you on the next catch-up.

No screenshots, no narrative, no timing. Just the rows.

Task 6 • Repeat for the real dealer extract when it arrives

Once Dany hands over the first real extract from PACCAR (after the credentials in [EgD-WGB-003](#) are provisioned), the extract will land in the source repository at `warranty-gene/dealer-records/peterbilt-atlantic/R01-halifax/vins/{VIN}.json` (and progressively R02 through R08). Repeat Task 3 with `rooftop_code` set to the source directory name. Repeat Task 4. Report the same four rows.

At that point Task 4.2's expected values will change — Dany will send the new expected values with the extract. Do not compute them yourself.

When to stop and message Dany

Anything that would block progress or that needs a decision only Dany can make:

1. **Task 1 access blocker.** The Azure subscription, resource group, or Key Vault is not accessible under Shrish's tenant.
2. **Task 3 schema mismatch.** A source record's JSON has a field type that the target's JSON column rejects (rare; both Postgres JSONB and SQL Server NVARCHAR(MAX) with ISJSON accept every shape in the sample).
3. **Task 6 record collision.** A real-extract record arrives with a VIN already loaded from the sample — which one wins is a Dany-side call.

Other questions can wait for the catch-up. If you took an initiative on the fork, note it in the Task 5 report with a link to the fork commit, and we'll walk through it together.

Provenance

- Source repository (all source authority): [EVEglyphDesign/hawkins-twin-platform](#)
- Schema (authoritative): [warranty-gene/schema/rich-target.schema.json](#)
- Canonical sample records: [warranty-gene/dealer-records/peterbilt-atlantic/R00-sample/vins/](#)
- Validation + indexing scripts (source-side, do not port): [warranty-gene/dealer-records/scripts/](#)
- Access map for real-data lane: [Warranty GENE Luke Access Prep \(EgD-WGB-003\)](#)